Cross-Family Latency Profiling of Object Detectors on an NVIDIA Tesla T4

DSAI Group Project*

Telecom Paris, Institut Polytechnique de Paris
May 2026

Abstract

Real-time object detection is limited less by arithmetic than by how that arithmetic maps onto the GPU. While CPU performance may have been the limiting factor at larger batch sizes, when running single images as a batch size, the factors that impact latency will include; what part of the computation executes first (and how those parts are fused), how many kernels are launched, and how much memory movement there is. None of these issues can be captured by a number of floating point operations (FLOPs) or frames-per-second (FPS). This paper presents a comparison of 24 different detectors, derived from 5 detector architectures (Two-Stages CNN, YOLO, DETR, RT-DETR, and a class of dense one stage detectors including RetinaNet, FCOS, and EfficientDet); run on a single NVIDIA Tesla T4, at FP32, with a single image processed at a time. Each architecture was then analyzed using the same protocol as before (640×640 COCO val2017 dataset at a batch size of 1) and ran through the same sequence of timing runs (1000 times) as previously mentioned. In the end we were able to identify three recurring patterns. First, it appears the convolutional layers dominate the execution time of most backbones (56% to 90%), while executing in FP32, the tensor cores remain unused. Second, during execution of the DETR family of models, there is wasted time (28% to 32%) used performing frozen batch normalization; however, since frozen batch normalization cannot be merged with the preceding convolutional layer operation, that portion of time remains "lost" as well. Lastly, due to falling back to python loops when it encounters the Deformable Attention Operator, Deformable DETR is the slowest model at 263 ms. We also provide mAP values for all 24 models listed above. Additionally we define each operator along with its respective potential acceleration(s), and map the bottlenecks identified in each family of models to specific levers to optimize. During our first optimization round we were able to achieve speedups ranging from 1.16 $\times$ to 2.34 $\times$ via FP16 auto cast alone, without any loss of accuracy. In our second optimization round we broke out the speeds achieved per-sub module. For example we found that convolutions and GEMMs could reach speedup ratios of up to 6.6 $\times$ when executed using FP16 Tensor Cores. Likewise we observed speedup ratios for fused activation functions such as SiLU up to 4.9 $\times$. Lastly we found that when executed using TensorRT's FP16 mode we were able to collapse up to 11 $\times$ of the number of per-image kernel launches compared to standard execution. Therefore, `compile` + auto cast was sufficient to take the DETR family of models to 3.2 $\times$ to 3.4 $\times$ while TensorRT's FP16 mode took RT-DETR R50/R101 to 5.1 $\times$ and 5.6 $\times$, respectively, while maintaining constant mAP. Then, based on these results, we extracted the operations that show the better improvement across every family. We turn that ranking into a design brief: a detector assembled from the fastest, most fusable blocks would be fast by construction. The deformable-attention fallback, selection, RoIAlign, and NMS remain the operations for which optimizations seems ineffective.

*Single team report aggregating the individual profiling and optimisation studies. Contributions by role: two-stage R-CNN (Charles Kayssieh); DETR family and shared tooling, P5 (Antoine Besson); RT-DETR family (Vansh); YOLOX/DAMO/YOLOv7, P3 (Jesus Sebastian); YOLOv5-11s (Yassine); RetinaNet, FCOS and EfficientDet, optimisation phase (Pacome).

1 Introduction and Problem Statement

The standard way to compare object detectors on COCO is to report mAP for accuracy and FPS or FLOPs for speed. The problem is that neither of those tells you *why* a given model runs at the speed it does. FLOPs in particular are not a reliable predictor of latency: they ignore memory bandwidth, the per-kernel launch overhead that accumulates at batch 1, and data-dependent operations like non-maximum suppression whose cost changes with the input.

We did not set out to produce another ranking of detectors by inference time. The goal was to break down each model's latency into its component operations - convolution, normalisation, attention, post-processing - and understand which part is actually responsible for the cost and why.

We chose batch size 1 both because it is the industry standard for latency benchmarks in deployment contexts and because it is the hardest case: with a single image there is no batching to hide per-kernel overhead, so every launch cost and memory transfer is paid in full on each image. For every model we measured end-to-end latency, then broke that time down by operator category (convolution, normalisation, activation, attention, pooling) and by hardware resource: GPU compute, memory transfers, kernel launches, and CPU/Python overhead. The entire baseline runs in FP32, which gives a clean, comparable starting point and makes the potential gain from mixed precision easy to quantify later. Our hardware is a single NVIDIA Tesla T4, a Turing-based GPU with 16 GB GDDR6, about 320 GB/s bandwidth, 2560 CUDA cores running at roughly 8 TFLOPS in FP32, and 320 Tensor Cores that are completely idle at FP32 but reach 65 TFLOPS in FP16.

2 Experimental Protocol

Every model receives one 640×640 RGB image at a time from the COCO val2017 set, in FP32 eager mode – no `torch.compile`, no TensorRT, no quantisation – at batch size 1. Before timing we run 50 warm-up iterations and discard them, which lets the cuDNN autotuner and the caching allocator settle. The actual measurement is 1000 forward passes, each bracketed by a `torch.cuda.Event` pair surrounding a call to `torch.cuda.synchronize()` to make sure the GPU has actually finished before we record the time. We report the median latency and compute FPS as $1000/\text{latency}$. Accuracy is the standard COCO $\text{mAP}^{50:95}$ computed with `pycocotools` over the full validation set. The random seed is fixed at 42 throughout, so every model sees the same images and the same proposal counts.

3 Model Families

The 24 models we benchmarked span five detector families. Table 1 gives the design context for each.

Table 1: Design context of the benchmarked families.

Family	Year	RT	Main innovation / type
Faster R-CNN	2015	no	Region Proposal Network; two-stage CNN
Mask R-CNN	2017	no	RoIAlign + mask branch; two-stage CNN
YOLOX/DAMO/v7	2021-23	yes	anchor-free, NAS, re-param; one-stage CNN
YOLOv5-v11	2020-24	yes	CSP backbone, NMS-free head (v10); one-stage CNN
DETR	2020	no	set prediction, no NMS/anchors; transformer
Deformable DETR	2021	no	sparse multi-scale attention; transformer
RT-DETR	2024	yes	efficient hybrid encoder; CNN+transformer
RetinaNet	2017	no	focal loss; dense one-stage CNN
FCOS	2019	yes	anchor-free, centre-ness; dense one-stage CNN
EfficientDet	2020	yes	BiFPN, compound scaling; one-stage CNN

Two-stage CNN. Faster R-CNN [16] proposes regions with an RPN, then classifies and refines each; Mask R-CNN [6] adds RoIAlign and a mask branch. The design targets accuracy over latency and was never meant to be real-time.

YOLO. The one-stage line [15] predicts boxes densely in a single pass for real-time use. The variants here differ in backbone and head: YOLOX is anchor-free [5], DAMO-YOLO uses a NAS backbone [25], YOLOv7 adds training-time bag-of-freebies [22], and the v5 to v11 series moves from CSP backbones [23] to the NMS-free one-to-one head of v10 [19, 21, 24].

DETR. DETR [2] reframes detection as set prediction with a transformer and bipartite matching, removing anchors and NMS. It is accurate but slow to converge and not real-time; Conditional [13] and DAB-DETR [12] speed convergence, and Deformable DETR [27] replaces dense attention with sparse multi-scale sampling.

RT-DETR. RT-DETR [26] is the first DETR built for real-time use, pairing a CNN backbone with a lightweight hybrid encoder and IoU-aware query selection, and reports beating YOLO on the speed-accuracy trade-off.

Dense one-stage (RetinaNet, FCOS, EfficientDet). These three predict boxes densely in a single pass but outside the YOLO line. RetinaNet [11] pairs a ResNet-FPN backbone with focal-loss classification and regression heads and a standard NMS. FCOS [18] drops anchors and predicts per-pixel boxes with a centre-ness branch. EfficientDet [17] uses an EfficientNet backbone, a weighted bidirectional feature network (BiFPN), and compound scaling. They round out the design space with dense one-stage CNNs that are neither anchor-free YOLO nor transformer. As it turned out, this family gives the cleanest results for the engine-level analysis in Section 10.

4 Elementary Building Blocks

Breaking a model into backbone, encoder, and head gives a useful overview, but it is not precise enough to decide what to optimise. Here we define the individual operators those blocks are built from, describe their cost, and explain what makes each easy or hard to accelerate. Table 2 covers the secondary operators; the four that dominated our measurements are discussed below.

Convolution. A 2D convolution computes

$$y_{o,i,j} = b_o + \sum_{c=1}^{C_{\text{in}}} \sum_{a,b=1}^{k} W_{o,c,a,b}\, x_{c,\,i+a,\,j+b}, \tag{1}$$

at a cost of about $HWC_{\text{in}}C_{\text{out}}k^2$ multiply-adds. The 1×1 case is a pure channel mixing, equivalent to a GEMM, and runs as `implicit_gemm/sgemm`. The 3×3 case is handled by the Winograd transform $F(2\times2, 3\times3)$, which cuts the multiplies per tile from 36 to 16 [10]. A depthwise convolution applies one $k\times k$ filter per channel, so its cost drops to $HWCk^2$ but its arithmetic intensity is low, which makes it bandwidth-bound [8]. Because convolution has high arithmetic intensity, it is the operator that responds best to acceleration: FP16/INT8 on the Tensor Cores gives a $2\times$ to $4\times$ band, and Conv-BN-activation fusion removes its surrounding element-wise traffic.

Normalisation. BatchNorm normalises with running statistics,

$$\mathrm{BN}(x) = \gamma\frac{x-\mu}{\sqrt{\sigma^2+\varepsilon}} + \beta = \underbrace{\frac{\gamma}{\sqrt{\sigma^2+\varepsilon}}}_{a} x + \underbrace{\Big(\beta - a\mu\Big)}_{c}, \tag{2}$$

which stabilises training [9]. At inference $\mu, \sigma^2, \gamma, \beta$ are constants, so (2) is an affine map and folds into the preceding convolution: $W' = aW$, $b' = ab + c$. After folding it costs nothing. FrozenBatchNorm is BN with the statistics frozen during training, which is standard in detection where batch sizes are small. It is equally foldable, but only if the implementation exposes it as a standard BN module – in the DETR family we found it runs as separate element-wise kernels instead, which is where the 28% to 32% overhead comes from. LayerNorm normalises each token over its feature dimension with *input-dependent* statistics, so it cannot be folded into a convolution and remains a real (if cheap) reduction; it is the normalisation used inside transformers [1].

Activations. ReLU $(\max(0,x))$, SiLU $(x\sigma(x))$ [14], and GELU $(x\Phi(x))$ [7] are element-wise and bandwidth-bound, so their cost tracks tensor size rather than FLOPs. ReLU is the cheapest (a compare-select); SiLU and GELU add a sigmoid or an erf/tanh and cost more per element. All three fuse into the epilogue of the preceding convolution or GEMM, so on a fusion-capable engine they are nearly free; a backbone that mixes two activation types (DAMO-YOLO) breaks that uniform fusion.

Attention. Scaled dot-product attention is

$$\mathrm{Attn}(Q,K,V) = \mathrm{softmax}\Big(\tfrac{QK^\top}{\sqrt{d}}\Big)V, \tag{3}$$

with $Q, K, V \in \mathbb{R}^{N\times d}$. The score matrix $QK^\top$ and the softmax are $\mathcal{O}(N^2 d)$ and $\mathcal{O}(N^2)$, so attention is quadratic in the token count N and, for long sequences, memory-bound on the $N\times N$ matrix [20]. Multi-head attention runs h smaller heads in parallel at the same total cost. This quadratic scaling is why a single low-resolution feature map (DETR, about 500 tokens) is cheap, while a multi-scale stack is not. Deformable attention removes the quadratic term: each query attends to only K learned sampling points,

$$\mathrm{DA}(q) = \sum_{k=1}^{K} A_{qk}\, W\, x\big(p_q + \Delta p_{qk}\big), \qquad K \ll N, \tag{4}$$

giving $\mathcal{O}(NKd)$ cost that makes multi-scale tractable [27]. The catch is the bilinear sample at the irregular offsets $p_q +$

Δp_{qk}: it needs a custom CUDA kernel, and without it the operator falls back to a Python loop. Dense attention accelerates well with IO-aware kernels like FlashAttention [3]; deformable attention is a different story and only gains speed if the compiled kernel is actually present.

Table 2: Secondary operators: role, cost, and ease of acceleration.

Operator	Role / cost	Accel.
FFN / MLP	token-wise $d \to 4d \to d$ linear pair; GEMM, $\mathcal{O}(Nd^2)$	easy
RoIAlign	bilinear-sample fixed grid per proposal; $\mathcal{O}(Ps^2)$ gather	hard
NMS	greedy IoU suppression; $\mathcal{O}(D^2)$, data-dependent, CPU	hard
Interp / reshape / concat	neck resize and layout; bandwidth-bound	medium
Residual add	$y = F(x) + x$ element-wise; bandwidth-bound	easy

Two operators from Table 2 are worth a closer look, since they drive the head latency in several families. RoIAlign samples a fixed $s \times s$ grid from each of P proposals with bilinear interpolation, so its cost grows with the proposal count and its gather pattern is irregular, which makes it hard to accelerate. NMS sorts detections and greedily removes overlaps above an IoU threshold; its cost is quadratic in the detection count, it is data-dependent, and it often runs partly on the CPU, which is why the NMS-free head of YOLOv10 is a structural speed-up rather than a kernel tweak.

5 Profiling and Optimisation Tools

Profiling. `torch.profiler` attributes wall time to named modules and to `aten` operators, with traces exported to Chrome/Perfetto. Nsight Systems reads the CUDA-API timeline directly (`cudaLaunchKernel`, `cudaMemcpyAsync`, `cudaStreamSynchronize`) and the physical kernel names, so it separates GPU compute from launch and transfer overhead at a per-launch cost near 0.1 μs. We used `torch.profiler` and Nsight Systems on every model in the study, including RT-DETR.

Optimisation levers (Sections 9 and 10). The operator breakdown suggested a fairly clear set of tools to try: mixed precision (FP16 autocast) and INT8 quantisation to wake the Tensor Cores on convolution and FFN; Conv-BN folding and Conv-BN-activation fusion to remove normalisation and activation traffic; CUDA graphs to cut the per-kernel launch cost at batch 1; a compiled deformable-attention kernel to eliminate the Python fallback; and graph-level export through TensorRT or ONNX Runtime to apply all of these automatically.

6 Per-Family Results at the Operator Level

Figure 1 splits each model by sub-module, Figure 2 and Table 3 split the backbone by operator. The point of this section is to show *which operator* causes each cost, more precisely than which block.

Table 3: Backbone time by operator, as % of backbone CUDA time. Norm is BatchNorm or FrozenBatchNorm, Act is ReLU/SiLU, Other is pooling, residual add and reshape. The dense one-stage row is a whole-model 3-model aggregate (the contributor reported aggregate operator shares, not a backbone-only split).

Model	Conv	Norm	Act	Other
DETR R50	60.5	27.8	10.8	0.9
DAB-DETR	56.2	31.8	11.1	0.9
Deformable DETR	61.8	27.8	9.5	0.9
RT-DETR R18	90.4	0.0	9.6	0.0
RT-DETR R50	82.4	0.0	17.6	0.0
YOLOX-m	77.4	14.0	7.5	1.1
YOLOX-x	86.7	7.4	5.2	0.7
DAMO-YOLO-M	75.1	14.8	8.7	1.4
yolov8s	90.4	0.0	1.9	7.7
RetinaNet/FCOS/Eff*	79.0	6.0	4.0	11.0

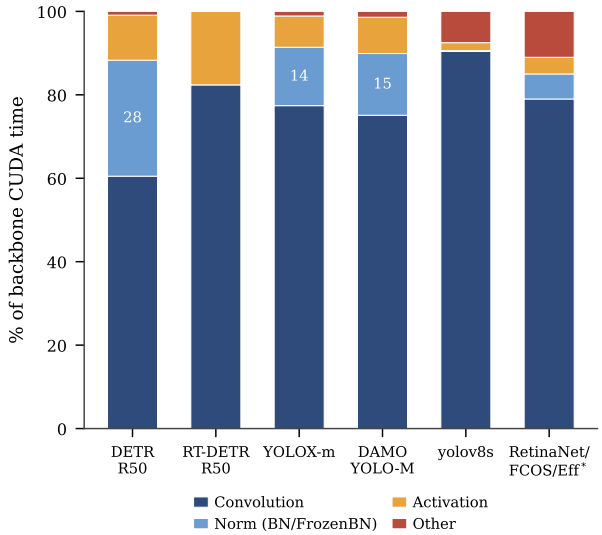

Figure 2: Backbone time by operation type (share of backbone CUDA time) for one representative model per family. Convolution dominates everywhere; the DETR family carries a large unfused FrozenBatchNorm block (28%) that the CNN detectors fold away. The last bar is the dense one-stage group (RetinaNet/FCOS/EfficientDet) as a whole-model 3-model aggregate.

Two-stage CNN. The backbone is roughly half the time and is convolution-bound; cuDNN folds its BatchNorm, so normalisation and activation are marginal. The cost the profiler attributes to the heads is RoIAlign and the per-proposal classifier plus NMS. This is where the V2 variant pays: its backbone is no heavier than the baseline, yet the RPN grows by 73% and the RoI head by 182%, because more proposals mean more RoIAlign gathers and a longer NMS. The bottleneck here is proposal-count-driven head work, not feature extraction.

DETR family. DETR, Conditional, and DAB are backbone-bound (65% to 71%), and the backbone splits as convolution (56% to 62%) plus an unfused FrozenBatchNorm block (28% to 32%, Table 3). The set-prediction transformer is cheap because it attends over a single low-resolution map of about 500 tokens, exactly the N in (3). Deformable DETR is the exception and the slowest model overall at 263 ms: its encoder is dominated by the deformable-attention operator of (4), which has no compiled kernel in this build and runs as a Python loop over a token sequence roughly 34$\times$ longer than DETR's. The cost is a missing kernel, not the architecture.

RT-DETR. The shallow R18 is balanced across encoder, decoder, and backbone; by R101 the backbone reaches 51%. Its backbone is almost pure convolution (82% to 90%) with no

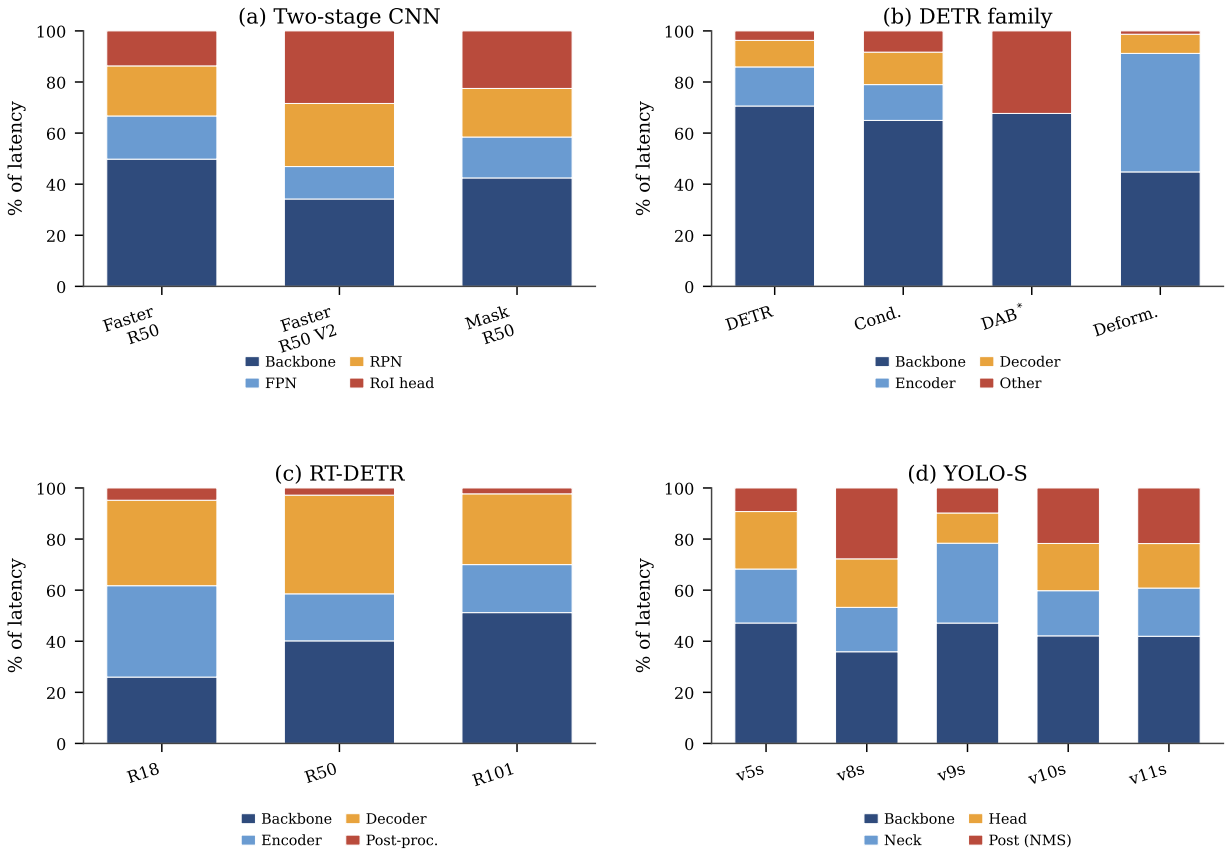

Figure 1: Latency share by sub-module, normalised to 100% per model. (a) Two-stage: backbone and the proposal/RoI heads; V2 shifts load onto a heavier RPN and RoI head. (b) DETR family: backbone-bound, except Deformable DETR where the multi-scale attention encoder dominates. (c) RT-DETR: the backbone share grows from 26% (R18) to 51% (R101). (d) YOLO-S: NMS post-processing is a real, content-dependent stage and is largest where the head keeps NMS. DAB-DETR's encoder and decoder cannot be separated (hook overlap) and are shown as one block.

separate normalisation, because the torchvision ResNet folds BN in eval mode. The ReLU share grows from 9.6% (R18) to 17.6% (R50) as the feature maps enlarge, the bandwidth signature of an element-wise operator scaling with tensor size.

YOLO. The CSP backbones are 75% to 90% convolution with small, mostly fused BatchNorm and a thin SiLU tail. The head tells the operator story: the NMS-bearing models spend a real, content-dependent slice in post-processing (v8s 28%, v11s 22%), whereas the NMS-free yolov10s replaces it with a cheap one-to-one decode, which also makes its latency the most stable of the group. v9s is the heaviest small model because GELAN/PGI issues about 37 200 convolution calls per pass, roughly double the rest, so at batch 1 its launch count, not its FLOPs, sets the latency.

Dense one-stage (RetinaNet, FCOS, EfficientDet). Aggregated across the three, the backbone is convolution-bound like the other CNN families: convolution is about 79% of the total time, with BatchNorm near 6%, activation near 4%, and the rest in pointwise and concatenation traffic (Table 3, last row). EfficientDet adds two operators the other families do not stress, the depthwise convolutions of its EfficientNet backbone and the weighted concatenations of its BiFPN neck, both of which matter at the engine level in Section 10. These three were profiled at the whole-model and per-operation level rather than split into backbone, neck, and head, so they do not appear as a panel in Figure 1; their operator profile is the last column of Figure 2.

7 Cross-Model Comparison

Table 4 and Figure 3 compare the models on latency, FPS, and mAP. Read together with the operator analysis, the ranking has clear causes rather than a single FLOP explanation.

Table 4: Per-image latency (ms, batch 1), FPS, and mAP$^{50:95}$ on COCO val2017, all 24 models.

Model	Latency	FPS	mAP
Two-stage CNN			
Faster R-CNN R50	49.0	20.4	0.131
Faster R-CNN R50 V2	98.6	10.1	0.174
Mask R-CNN R50	70.7	14.2	0.134
DETR family			
DETR R50	77.8	12.9	0.424
Conditional DETR	82.7	12.1	0.385
DAB-DETR	85.1	11.7	0.398
Deformable DETR	263.3	3.8	0.410
RT-DETR			
RT-DETR R18	24.5	40.8	0.470
RT-DETR R50	57.4	17.4	0.534
RT-DETR R101	90.1	11.1	0.548
YOLO			
YOLOX-m	21.8	45.9	0.475
DAMO-YOLO-M	25.3	39.5	0.510
YOLOv7	33.9	29.5	0.519
YOLOX-l	41.8	23.9	0.505
YOLOv7-X	60.7	16.5	0.537
YOLOX-x	75.6	13.2	0.523
YOLO-S			
yolov5s	13.1	76.3	0.425
yolov8s	14.4	69.5	0.444
yolov10s	15.8	63.1	0.456
yolov11s	16.6	60.3	0.464
yolov9s	26.0	38.4	0.460
Dense one-stage			
RetinaNet R50	49.8	20.1	0.378
FCOS R50	50.8	19.7	0.336
EfficientDet D4	109.2	9.2	0.448

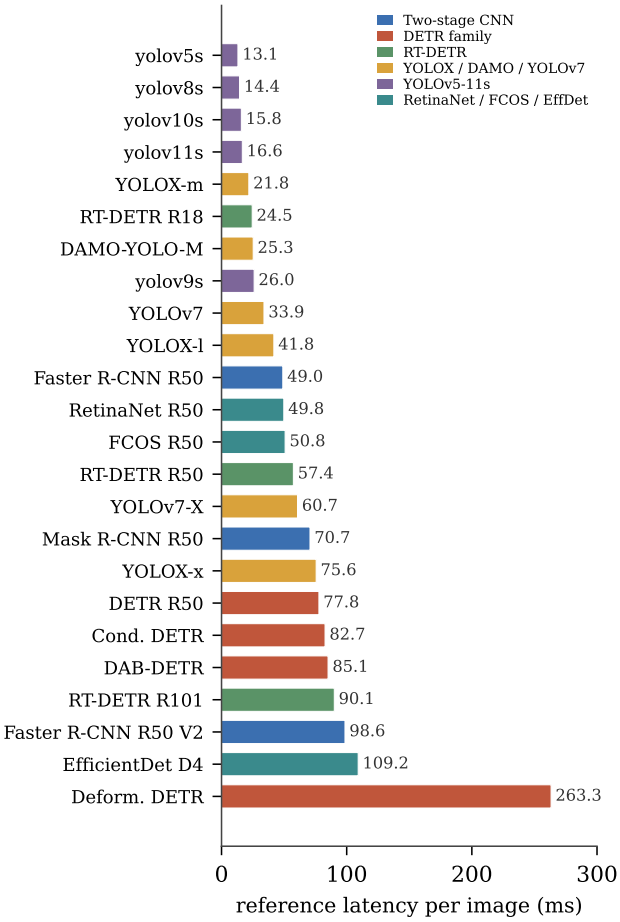

Figure 3: Per-image latency, sorted, coloured by family. Deformable DETR is the clear outlier, set apart by a missing operator kernel rather than by model size.

The accuracy-per-millisecond picture is consistent. RT-DETR holds the best trade-off in the set at 0.534 mAP and 57 ms. Its backbone is fully folded convolution and its hybrid encoder keeps the token count low. The small YOLO models cluster at 13 ms to 26 ms with 0.42 to 0.46 mAP and owe their stability to fused Conv-BN-SiLU backbones; the NMS-free yolov10s is both fast and the most deterministic. The DETR family pays for unfused normalisation, and Deformable DETR pays an order of magnitude for one un-compiled operator. The two-stage models sit mid-pack on speed but lowest on mAP in this run, with their head cost driven by RoIAlign and NMS rather than the backbone. The dense one-stage group sits across the same range: RetinaNet (49.8 ms, 0.378) and FCOS (50.8 ms, 0.336) land beside Faster R-CNN R50, while EfficientDet D4 is the second slowest model in the study at 109 ms despite its efficiency-oriented design, because its depthwise-heavy backbone and BiFPN run unfused in eager FP32 and are the ones the engine later collapses most (Section 10).

8 Optimisation Levers per Family

Table 5 maps each family's measured operator bottleneck to the lever that addresses it, ordered by expected payoff on this hardware.

Table 5: Operator bottleneck to optimisation lever, with the measured evidence.

Family / operator evidence	Lever
Two-stage: conv backbone + RoIAlign/NMS heads; V2 head +182%	Pinned input; CUDA graphs / TensorRT; fewer proposals
DETR: Conv 56-62%, FrozenBN 28-32% unfused, all FP32	Conv+BN fold (free); FP16 autocast for Tensor Cores
Deformable DETR: deformable-attention Python fallback, 263 ms	Build the CUDA MSDA kernel; then FP16
RT-DETR: Conv 82-90%, ReLU grows to 18%	FP16/INT8; lighter backbone; Conv-BN-ReLU fusion
YOLO: Conv 75-90%; NMS data-dependent; v9s 37k conv calls	Re-parameterisation [4]; CUDA graphs; NMS-free head; INT8
Dense one-stage: Conv ≈79%, depthwise + BiFPN; NMS dynamic	`compile`/TorchScript + FP16; constant-fold then zone TensorRT; INT8

The recurring lesson is that the bottleneck is rarely the arithmetic the model was designed around. Convolution dominates compute but responds to precision and fusion; the remaining time is normalisation and activation traffic that fusion deletes, launch overhead that CUDA graphs collapse, and post-processing that a different head removes. FLOP reduction is the least useful lever of the set at batch 1.

9 First Optimisation Experiments

This round applies the cheapest and most portable levers: Conv-BN folding, FP16 autocast (AMP), `torch.compile`, and manual CUDA graphs. It now covers every family. The heavier levers (TensorRT, ONNX Runtime, INT8, a compiled deformable-attention kernel) are treated in Section 10. The goal is to explain each outcome, not only to report a speed-up: which operators moved to FP16, which kernels fused, and what stayed blocking. Table 7 is that explanation; Table 6 and Figure 4 are the numbers.

Which operators move to FP16, which stay FP32. AMP casts the compute-bound operators (Conv2d, Linear, the attention-projection GEMMs) to FP16, so they dispatch to the Turing Tensor Cores (about 65 against 8 TFLOPS FP32). The numerically sensitive reductions (BatchNorm, softmax, LayerNorm, the RoIAlign coordinate maths) stay in FP32. This split is why every measured mAP moved by less than 10^{-3}.

Table 6: First-round results: in-session FP32 baseline (ms), best optimised latency (ms), speed-up, best configuration, and $mAP^{50:95}$. YOLO-S uses per-brick lever selection (fold everywhere, FP16 only where it helps); its mAP is the optimised value, preserved or marginally higher than FP32. yolov6s is omitted (no semantic baseline). For the dense one-stage group, comp+AMP is `torch.compile`+autocast and TS+AMP is TorchScript+autocast (the winner on EfficientDet).

Model	Base	Opt	×	Config	mAP
Two-stage CNN					
Faster R-CNN R50	48.5	24.7	1.97	comp+AMP	0.131
Faster R-CNN R50 V2	89.2	44.3	2.01	comp+AMP	0.174
Mask R-CNN R50	68.9	55.4	1.24	comp+AMP	0.134
DETR family (AMP mAP pending)					
DETR R50	88.1	37.7	2.34	AMP+fold	0.424
Conditional DETR	94.0	40.7	2.31	AMP+fold	0.385
DAB-DETR	96.0	45.1	2.13	AMP+fold	0.398
Deformable DETR	291.3	202.7	1.44	AMP+fold	0.410
RT-DETR					
RT-DETR R18	24.5	19.8	1.24	compile	0.470
RT-DETR R50	57.4	44.9	1.28	compile	0.534
RT-DETR R101	90.1	76.9	1.17	AMP	0.548
YOLO					
YOLOX-m	21.8	18.8	1.16	AMP	0.475
YOLOX-l	41.8	22.0	1.90	AMP	0.505
YOLOX-x	75.6	33.4	2.26	AMP	0.523
YOLOv7	33.9	17.9	1.90	AMP	0.519
YOLOv7-X	60.7	26.6	2.29	AMP	0.537
DAMO-YOLO-M	25.3	15.0	1.69	comp+AMP	0.510
YOLO-S					
yolov5s	13.0	9.2	1.42	fold+FP16	0.434
yolov8s	14.0	9.8	1.43	fold+FP16	0.452
yolov9s	25.1	22.1	1.13	fold+FP16	0.469
yolov10s	15.5	11.3	1.37	fold+FP16	0.469
yolov11s	17.3	11.4	1.52	fold+FP16	0.469
Dense one-stage					
RetinaNet R50	49.8	21.7	2.29	comp+AMP	0.377
FCOS R50	50.8	23.9	2.13	comp+AMP	0.334
EfficientDet D4	109.2	30.5	3.57	TS+AMP	0.448

What accelerated, and why. FP16 autocast is the one universal win: 1.16× to 2.34× at no accuracy cost. The gain tracks how much convolution the backbone carries. Small models gain least or regress (YOLOX-m 1.16×, RT-DETR R18 0.85×): the cast and copy traffic, visible as thousands of `aten::copy_` calls, is not amortised by enough FP16 compute. Large models gain most (YOLOX-x 2.26×, YOLOv7-X 2.29×, DETR R50 2.34×) because dense convolution dominates and runs on the Tensor Cores. `torch.compile` adds a smaller, separate win: it fuses element-wise chains (Conv+ReLU, Add+ReLU) and cuts per-kernel launch overhead, which matters most at batch 1. That is what took Faster R-CNN R50 to nearly 2× (48.5 to 24.7 ms) and tamed DAMO-YOLO's launch-bound variance.

Conv-BN folding: one lever, opposite outcomes. On the CNN backbones (YOLO, two-stage R50) the fold is a wash (0.95× to 1.01×, a slowdown on Mask R-CNN): cuDNN's BN kernel is already inference-optimised and the dominant cost is reading the feature map, which folding does not remove. On the DETR family the same fold saves a real 6.4 to 7.2 ms (1.07× to 1.08×): DETR's custom `FrozenBatchNorm2d` is not recognised by `fuse_conv_bn_eval`, so it ran as separate element-wise kernels. We wrote the fold by hand and validated it to 10^{-4}. Folding also removes the FP32 islands from the backbone, so AMP then stacks better than multiplicatively (DETR R50: 1.085×1.55 expected, 2.34× measured). Fusion helps only where the framework was not already fusing.

The YOLO-S recipe. Each of the 23 to 29 bricks per model

Table 7: Lever support matrix from the first optimisation round: what each lever targets, where it worked, and where it was blocked or unsupported by the tool.

Lever	Targets / effect	Worked on	Blocked or unsupported on
AMP FP16 (autocast)	casts Conv2d, Linear, attention GEMMs to FP16 Tensor Cores; reductions kept FP32	all families, 1.16 to 2.34×; gain scales with backbone compute	Mask R-CNN mask head crashes (FP16 type mismatch); RT-DETR R18 regresses to 0.85× (cast/copy > gain)
Conv-BN / FrozenBN fold	absorb the BN affine into the preceding convolution	DETR FrozenBN (≈7 ms, manual); two-stage V2 FPN BN (10%); YOLO-S aggregation bricks	wash on cuDNN-optimised CNN BN (YOLO 0.95-0.97×, Mask R50 slower); `fuse_conv_bn_eval` does not support `FrozenBatchNorm2d`
`torch.compile` / Inductor	elementwise fusion (Conv+ReLU, Add+ReLU), lower dispatch overhead	Faster R-CNN, RT-DETR (1.14 to 1.28×), DAMO-YOLO (1.05×)	YOLOv7/v7-X flat `nn.Sequential` (no hierarchy); DETR graph-breaks (NestedTensor, dynamic box count)
Manual CUDA graphs	replay a static forward pass to cut launch overhead	attempted on small/launch-bound models	YOLOX-m dynamic control flow; RT-DETR CPU→CUDA copy inside the HF forward
ninja MSDA / TensorRT / ONNX / INT8	compiled and quantised kernels, plugin-level fusion	second round: Section 10 (RT-DETR TensorRT, DETR compile+AMP)	Deformable DETR Python fallback is the residual 203 ms target

was measured under fold, FP16, and fold+FP16, and the best lever kept per brick. The fold wins on most bricks; FP16 pays only on the plain convolutions of the newer backbones and regresses on the small v5s tiles that under-fill the Tensor Cores. Median gains are a moderate 1.13× to 1.52×, but the real effect is the tail: the FP32 baselines were 60 to 82% host-bound, and removing per-op dispatch collapses p99 from up to 601 ms to 18 ms (v5s, 33×) and the run-to-run std by up to 36×. For single-image deployment, that determinism matters more than the median.

What stayed blocking. Four things resisted the cheap levers. The deformable-attention operator has no compiled kernel and runs as a Python loop; it is not a Tensor-Core GEMM, so FP16 cannot touch it, and Deformable DETR plateaus at 1.44× and 203 ms. RoIAlign and NMS are memory-bound and irregular; they neither fuse under `torch.compile` nor benefit from FP16, which is why Mask R-CNN stops at 1.24×. AMP crashes the Mask R-CNN evaluation outright (an FP16 type mismatch in the mask head). And the graph-capture levers failed on architecture grounds: CUDA graphs broke on YOLOX-m's dynamic control flow and on a CPU-to-CUDA copy inside the HuggingFace RT-DETR forward, while `torch.compile` found no module hierarchy in YOLOv7's flat `nn.Sequential` and graph-breaks on DETR's NestedTensor and dynamic detection count.

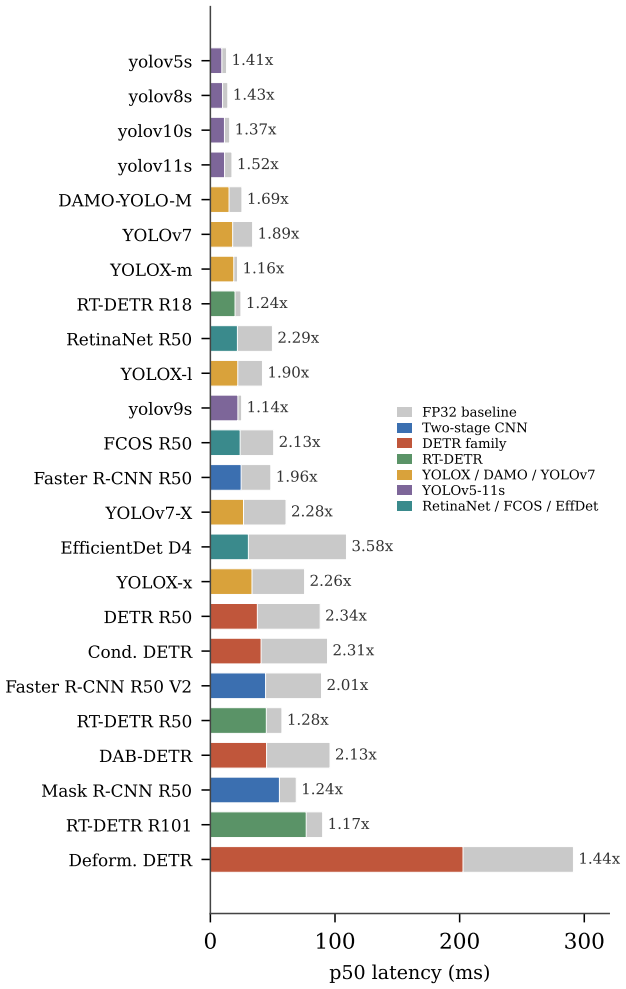

Figure 4: FP32 baseline (grey) against the best optimised configuration (colour) per model, with the speed-up. Baselines are re-measured in-session, so they differ slightly from Table 4; relative speed-ups are unaffected.

10 Acceleration per Sub-Module

Whole-model speed-ups hide where the gain comes from. This section answers the direct question: take a model before and after acceleration, look at each sub-module (convolution, BatchNorm, activation, GEMM, post-processing), say how much each one gained, and explain why. The dense one-stage family (RetinaNet R50, FCOS R50, EfficientDet D4) contributes the cleanest before/after case. EfficientDet is the one model in the whole study where a full-model TensorRT engine compiles, so the baseline and the engine share one profiler trace

and the per-operator gains are exact.

Two backends carry the gains. FP16 autocast (AMP) casts the compute-bound operators to the Tensor Cores. TensorRT goes further: it folds Conv+BN, fuses the activation into the convolution, runs every layer it can in FP16, and replaces hundreds of small kernels with one engine. The cost it does not touch is data-dependent post-processing. The families that ship a TensorRT engine (RT-DETR, YOLO-S, the YOLOX/DAMO/YOLOv7 group, EfficientDet) reach the largest numbers. The two-stage CNNs and the DETR family do not: a torchvision or HuggingFace dependency conflict blocked the engine build, so their best lever is `compile`+AMP, and we report only that.

Two-stage CNN, per operator class. Table 8 gives the best factor per class. The headline numbers are two different mechanisms. AMP moves the convolution and GEMM time to the Tensor Cores: 3.3× to 7.6× on that class, with the V2's 67.3 ms collapsing to 8.9 ms, the largest absolute cut in the study (58.4 ms). `torch.compile` accelerates the small, launch-bound classes instead: RoIAlign 4.4×, NMS 4.2×, sort/top-k 5.8× on Faster R-CNN, and 37.9× on the V2 memory class (6.9 to 0.2 ms). What it removes is dispatch and allocator traffic, not arithmetic. The element-wise class is the consistent loser: AMP regresses it to 0.56× to 0.80× through cast traffic, and on the dynamic graphs (V2, Mask) compile makes it worse still (down to 0.32×) by generating Triton kernels that recompile as the proposal-dependent shapes change. Mask R-CNN gains little outside convolution: its RoIAlign barely moves (1.02×).

Table 8: Two-stage CNN: best speed-up per operator class and the lever that achieves it (A = AMP, C = `torch.compile`, F = Conv-BN fold). Baseline class times in ms in the second column (Faster / V2 / Mask).

Operator class	Base (ms)	Faster	V2	Mask
Conv & GEMM	30/67/40	3.81 (A)	7.55 (A+F)	3.26 (A)
Activ. & norms	14/10/20	1.02 (F)	1.00 (none)	1.37 (C)
RoIAlign	2.5/2.7/3.3	4.37 (A+C)	2.27 (A+C)	1.02 (F)
NMS	0.2/0.2/0.2	4.24 (A+C)	2.29 (A+C)	1.01 (F)
Sort / top-k / gather	1.9/2.1/2.8	5.78 (A+C)	3.09 (A+C)	1.35 (C)
Other / memory	0.5/6.9/2.4	3.42 (C)	37.9 (C)	1.00 (none)

YOLOX / DAMO / YOLOv7, per operator. This group was profiled operator by operator under AMP, `compile`, and TensorRT (Table 9). Reading it needs one rule: many per-op cells below 1.0× are accounting artifacts, not regressions. When Conv+BN folds, the BatchNorm time moves into the convolution row, so the bare convolution reads slower while the Conv+BatchNorm budget is unchanged. The trustworthy rows are `Conv_total` (3×3 plus 1×1 summed, nothing folds in or out of it), SiLU, and MaxPool. They tell a clean, two-step story. AMP roughly doubles the bandwidth-bound activations: SiLU averages 2.22×. TensorRT roughly doubles that again by fusing SiLU into the preceding convolution and deleting the kernel: SiLU averages 4.53×, peaking at 4.88× on YOLOv7-X. The convolution class behaves by model size. On the smallest model (YOLOX-m) AMP gives little because the tiny convolutions do not fill the Tensor Cores, and the real win waits for TensorRT (1.91×). On the largest (YOLOX-x, YOLOv7) AMP alone already reaches 1.81× to 1.96× on `Conv_total`, and TensorRT lifts it to 3.15× and 5.00×, the strongest convolution acceleration in the study. `compile` fails to trace the flat `nn.Sequential` of YOLOv7 and v7-X, so its column is the eager result, which is why the missing per-submodule profile flagged in the previous round is now filled by AMP and TensorRT rather than by `compile`.

Table 9: YOLOX/DAMO/YOLOv7: cumulative operator speed-up vs the FP32 baseline. "fused" means the operator no longer exists as a separate kernel inside the TensorRT engine; "op." marks the compile-first DAMO graph that hides its middle stages.

Model	Operator	AMP	compile	TensorRT
YOLOX-m	Conv_total	0.72	op.	1.91
	SiLU	1.95	op.	fused
YOLOX-l	Conv_total	1.43	op.	2.67
	SiLU	2.29	op.	fused
YOLOX-x	Conv_total	1.81	op.	3.15
	SiLU	2.73	op.	fused
YOLOv7	Conv_total	1.96	1.84	5.00
	SiLU	2.11	2.11	4.18
YOLOv7-X	Conv_total	2.31	2.16	4.54
	SiLU	2.03	2.07	4.88
DAMO-YOLO-M	Conv_total	op.	op.	1.06
	Conv1×1	op.	op.	2.16

DETR family, and why CUDA graphs flip the AMP rule. Table 10 is the full technique matrix, re-measured back to back on one session. The headline changes the round-one advice. In eager mode the per-op analysis said to scope AMP to the backbone, because a blunt full-model cast regresses DAB-DETR (0.90×): the FP16/FP32 conversions inserted around the decoder's small head matmuls cost more than the Tensor Cores save. That holds in eager, where the best config is the scoped `fold+chlast+amp-backbone` at 1.34× to 1.72×. But once `torch.compile` captures a whole-model CUDA graph, the rule inverts: full FP16 now beats scoped FP16, and `compile`+AMP is the clear winner on every model (3.20× to 3.41× on the three small models, 2.16× on Deformable). The head-Linear FP16 penalty was never an arithmetic cost; it was launch overhead, and the graph capture removes it, so the tiny FP16 head matmuls stop hurting. The fold itself is a fixed 1 to 4 ms of FrozenBN kernels, so its factor shrinks with model size (1.10× on Conditional, 1.03× on Deformable, whose time sits in the deformable attention). `cudnn.benchmark` is a no-op (0.95× to 1.00×). The hand-stacked `superfast` config lands at 1.67× to 1.77×, behind plain `compile`+AMP, because it compiles the head without CUDA graphs to dodge a capture crash and so gives up exactly the whole-model graph that makes the simple config win. TensorRT was not available here (installing `torch_tensorrt` breaks torchvision in this environment), so `compile`+AMP is the DETR deployment lever.

Table 10: DETR family: speed-up per technique vs the in-session FP32 baseline (36.7, 42.7, 44.7, 130.5 ms p50). "bb" = scoped to the backbone; chlast = channels-last.

Lever	DETR	Cond.	DAB	Deform.
convbn fold	1.02	1.10	1.08	1.03
autocast, full model	1.07	1.08	0.90	1.29
autocast, bb only	1.22	1.26	1.07	1.25
convbn + bb autocast	1.61	1.65	1.30	1.36
convbn + chlast + bb autocast	1.64	1.72	1.34	1.39
`cudnn.benchmark`	0.95	1.00	0.99	1.00
`compile`	1.17	1.23	1.21	1.27
`superfast` (hand)	1.77	1.76	1.67	1.39
`compile` + AMP	**3.24**	**3.41**	**3.20**	**2.16**

YOLO-S, per stage and into the engine. Table 11 rolls the per-brick winners up to backbone, neck, and head, and adds

the TensorRT FP16 engine that now exists for the family. The PyTorch island build (fold plus selective FP16) gives a moderate 1.13× to 1.52×; the engine then nearly doubles that again (1.78× to 2.88× over the FP32 baseline) because it applies the same two levers everywhere and removes the launches the island build could only fold, not delete. The clearest single case is v9s: the island build barely moved it (1.13×) because its heavy reparameterisable GELAN neck resists a blanket FP16 cast, but TensorRT fuses that neck end to end and reaches 2.88×, the family's best. The deployment payoff is the tail more than the median: the FP32 baselines were 60 to 82% host-bound, and the static engine collapses their p99 from up to 601 ms to under 9 ms (44× to 80×) and the run-to-run std to under 0.11 ms, at 100% GPU utilisation. mAP is preserved or marginally higher on every trained model (v5s 0.425 to 0.438, v9s 0.460 to 0.476), within subset noise. v6s is latency-only (random-init weights), so it carries no baseline factor.

Table 11: YOLO-S: stage-level speed-up from the per-brick island build (fold + FP16), and the TensorRT FP16 engine speed-up over the same FP32 baseline. v6s has no FP32 baseline.

Model	B.bone	Neck	Head	Island	TRT FP16
v5s	1.42	1.35	1.34	1.42	1.78
v6s	2.13	1.22	1.02	1.67	–
v8s	1.22	1.08	1.02	1.43	1.86
v9s	2.02	1.95	1.28	1.13	2.88
v10s	1.34	1.41	2.48	1.37	–
v11s	1.44	1.44	1.72	1.52	2.29

RT-DETR, per backend. Table 12 extends the family to graph-level export. The levers must be combined: AMP alone regresses R18 (0.85×) and is neutral on R50 (1.01×) because the eager cast and copy traffic eats the gain, while `compile` fuses the casts into the kernels and the pair reaches 2.49× to 3.37×. TensorRT FP16 is best for the two heavier models (5.06× and 5.57×): Myelin fuses convolution, GEMM, and reformat into one engine. mAP is stable throughout; the largest change is R101 at 0.548 to 0.547. Nsight on the TensorRT runs shows what is left per stage: the candidate-selection (`node_select`) step at a consistent 14 to 15% of execution, host-to-device copies of about 1 ms per image, and `cudaStreamSynchronize` as the dominant API call. The next levers for this family are therefore not convolution work but INT8, GPU-resident I/O, and a cheaper query selection.

Table 12: RT-DETR: latency in ms (speed-up) per backend. PyTorch eager FP32 is the baseline; mAP unchanged except R101 TensorRT FP16 (0.548 to 0.547).

Backend	R18	R50	R101
eager FP32	24.5	57.4	90.1
`compile` FP32	19.8 (1.24)	44.9 (1.28)	79.3 (1.14)
AMP FP16	28.7 (0.85)	57.1 (1.01)	76.9 (1.17)
`compile`+AMP	9.8 (2.49)	17.7 (3.24)	26.7 (3.37)
ORT+TensorRT FP32	15.6 (1.57)	34.8 (1.65)	61.9 (1.45)
ORT+TensorRT FP16	10.4 (2.36)	11.3 (5.06)	16.2 (5.57)

Table 13 splits that engine gain by operator. Three things move at once. Convolution gains 1.69× to 3.52×, more on the deeper backbone because its larger convolution work amortises the eager dispatch better. GEMM gains the most, 2.11× to 6.62×, because TensorRT swaps the FP32 `volta_sgemm` kernels for FP16 Tensor-Core GEMMs and fuses bias, residual, and ReLU into them. And the kernel launch count drops 8.7× to 11.5× (1684 to 147 launches per image on R101), which is the reason the end-to-end speed-up exceeds the convolution-only or GEMM-only number: fewer kernels means fewer intermediate tensors written to memory. What stays is the candidate selection (`node_select`): 1.89 ms and 16.6% of latency on R50. It is top-k, scan, and gather work, not a dense matmul, so the Tensor Cores do not help it.

Table 13: RT-DETR, eager FP32 to ORT+TensorRT FP16 per operator: convolution and GEMM time speed-up, kernel-launch reduction, and the residual `node_select` cost.

Model	Conv	GEMM	Launches	node_select
R18	1.69	2.11	8.73	1.03 ms (9.9%)
R50	2.69	4.12	10.53	1.89 ms (16.6%)
R101	3.52	6.62	11.45	1.88 ms (11.6%)

EfficientDet, FCOS, RetinaNet: the clean before/after. EfficientDet D4 is the one model where a full-model TensorRT engine compiles, so its profiler trace shows exactly which baseline kernels vanish (Table 14, Figure 5). The engine deletes the convolution kernels (1517 ms over the profiling window), the BatchNorm kernels (989 ms, folded into the convolutions), the depthwise convolutions (960 ms), the FP32 GEMM (930 ms), and both halves of SiLU (1502 ms, fused as the convolution epilogue). Even the BiFPN concatenations (516 ms) mostly disappear because TensorRT writes the producer kernels straight into the concatenated buffer instead of copying. All of that work moves into one `tensorrt::execute_engine` call plus a generated pointwise kernel. Pure CUDA time drops 1.62× and wall-clock 2.32×; the gap between the two is the Python dispatch the engine removes. mAP holds at 0.4477 to 0.4474.

Table 14: EfficientDet D4, baseline to full TensorRT FP16. Times are ms over the 150-pass profiling window; "why" is the mechanism that removes the kernel.

Baseline kernel	Base	TRT	Why
cudnn convolution	1517	0	FP16 fused conv kernel
cudnn batch_norm	989	0	folded into conv
depthwise conv	960	0	FP16 depthwise tactic
volta_sgemm (FP32)	930	0	FP16 Tensor-Core GEMM
SiLU (+ kernel)	1502	0	fused as conv epilogue
cat (BiFPN concat)	517	0.7	producer writes, no copy

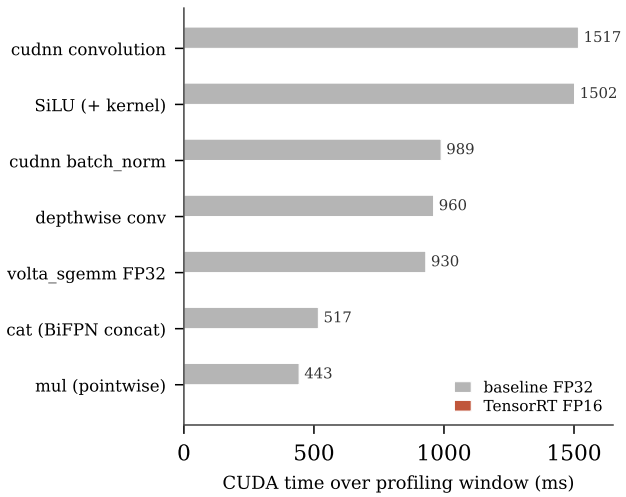

Figure 5: EfficientDet D4, the dominant baseline kernels (grey) against what survives the TensorRT FP16 engine (red). The convolution, BatchNorm, depthwise, GEMM, and SiLU kernels are gone; one fused engine call replaces them.

FCOS and RetinaNet show the opposite case: their full-model engine build fails. The NMS emits a variable number of boxes, so the export hits an unbounded dynamic shape and TensorRT crashes. The clean route is to compile only the static zone (backbone, FPN, dense head) and leave NMS in eager. With a constant-fold pass first, that zone reaches 1.93× on FCOS and 1.66× on RetinaNet at no accuracy cost. Their best overall lever is the same as the DETR and two-stage families: `compile`+AMP (2.13× and 2.29×). The lesson is concrete: the dense convolutional part accelerates cleanly, and the data-dependent post-processing is what keeps a model out of a single engine.

What stays blocking, across the section. The same operators resist every lever. The element-wise class regresses under a blunt AMP cast wherever it was measured (0.56 to 0.80× on the two-stage models, the DAB-DETR full cast at 0.90×, RT-DETR R18 at 0.85×), and only a graph that fuses the casts away recovers it. RoIAlign on Mask R-CNN does not move (1.02×). The deformable-attention fallback caps its model at 1.44×. The RT-DETR `node_select` and the two-stage and FCOS/RetinaNet NMS stay on the host because their shapes are data-dependent. These are the floor that the next section reads as a design constraint.

11 The Most Accelerable Operations Across Families

With every family measured, one transversal question can be answered: which elementary operations respond best to FP16 and to TensorRT fusion, regardless of which model they sit in. Table 15 ranks them by the best speed-up we measured, and Figure 6 plots the same data. The pattern is consistent with the operator theory in Section 4: the dense, high-arithmetic-intensity operators win, the bandwidth-bound ones win once fused, and the data-dependent ones do not move.

Table 15: Elementary operations ranked by best measured speed-up, with the lever and where it was seen. The lower block is the floor: operations that resist FP16 and fusion.

Operation	Lever / where	Best ×
GEMM / Linear / 1×1 conv	TRT, RT-DETR R101	6.62
Conv+BN+activation (fused)	TRT, YOLOv7	5.00
SiLU / pointwise activation	TRT, YOLOv7-X	4.88
Convolution (alone)	TRT, RT-DETR R101	3.52
Pointwise chains (mul/add)	TRT, EfficientDet	≈3–5
Pooling (MaxPool)	TRT, cross-model	2.65
Depthwise convolution	TRT, EfficientDet	≈2–3
Concat	TRT, EfficientDet	eliminated
node_select / top-k	— (not dense)	≈1.0
NMS	— (host, dynamic)	≈1.0
RoIAlign	— (irregular gather)	1.02
Deformable attention (fallback)	— (no kernel)	1.44 cap
Upsample (FP16)	regresses	0.75–0.92

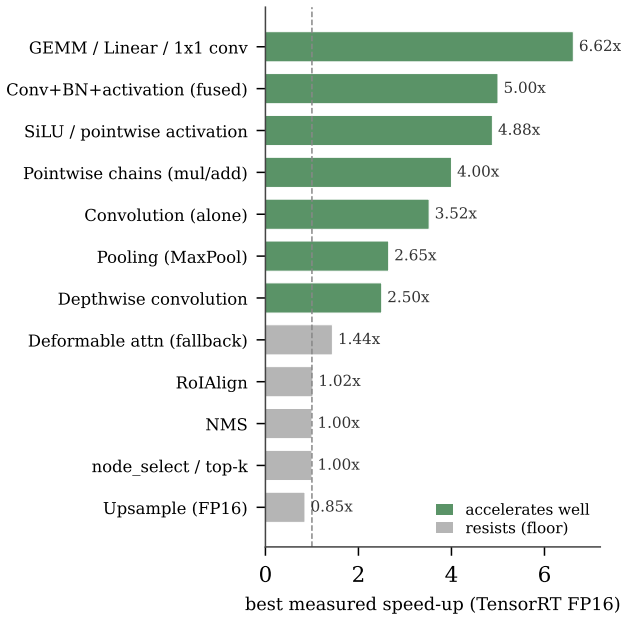

Figure 6: Best measured speed-up per elementary operation, coloured green where the operation accelerates well and grey where it resists. The dense and fusable operators sit at the top; the data-dependent post-processing sits at the floor near 1.0×.

Three mechanisms explain the top of the list. First, precision: GEMM, convolution, and the 1×1 projections are high arithmetic intensity, so moving them from FP32 to the FP16 Tensor Cores is a near-free 2× band, confirmed by the cross-model averages (`Conv_total` 1.65× under AMP, 3.06× under TensorRT). Second, fusion: the bandwidth-bound operators (SiLU, ReLU, pooling, residual adds) cost memory traffic, not math, so folding them into the preceding convolution as an epilogue deletes the kernel entirely. SiLU is the clearest example, doubling under FP16 (2.22× average) and doubling again under TensorRT fusion (4.53× average). Third, launch removal: at batch 1 the engine collapses hundreds of kernel launches into one, up to 11.5× fewer on RT-DETR R101, which is why the whole-model speed-up beats any single operator's.

The floor is just as consistent. Selection, top-k, NMS, and RoIAlign are data-dependent or irregular: their shapes change per image, they gather from scattered memory, and they often touch the host. No precision or fusion lever reaches them. The deformable-attention fallback is the extreme case, a missing kernel that caps its model at 1.44×. These operations are not slow because of arithmetic; they are slow because they break the static, dense, fusable pattern that the Tensor Cores and the engine are built for.

12 Toward a Detector Built From Accelerable Parts

The ranking in Section 11 is usually read as an optimisation guide for existing models. It can also be read forwards, as a design brief. If the fast operations are known, a future detector can be assembled mostly from them, so that it is fast not because it was optimised after the fact but because almost all of it compiles into a single FP16 (later INT8) engine. The components below are exactly the ones each family contributed as its best-accelerating block.

A backbone of regular Conv+BN+activation blocks, or MBConv blocks (depthwise plus pointwise plus SiLU, as in EfficientNet and MobileNetV3), gives the engine the fused Conv-

BN-SiLU kernels that topped the table (up to 5.00×). Keeping the activation to one fusable family (ReLU, SiLU, GELU) preserves uniform fusion, which the mixed-activation DAMO-YOLO backbone showed is easy to break. A static neck, a plain FPN or a BiFPN with its mixing coefficients frozen at inference, lets constant folding run and keeps the whole neck inside the engine; on FCOS and RetinaNet that single change took the static zone from neutral to 1.66× to 1.93×. A dense head in the style of FCOS, CenterNet, or the NMS-free YOLOv10 one-to-one head avoids the proposal-driven RoIAlign and the quadratic NMS that pin the two-stage models to 1.24×. Where NMS is unavoidable it should run as a GPU-native operation with a fixed top-K bound, so the output shape is static and the engine does not fall back to the host. And every grid, anchor, and index should be a pre-computed GPU buffer, never a CPU tensor, since CPU tensors are what broke CUDA-graph capture on FCOS and RetinaNet.

The two things to avoid are the two clearest floor cases of this study. An operator with no compiled kernel, the deformable-attention fallback, costs an order of magnitude and cannot be reached by any precision or fusion lever; an architecture that needs it should ship the kernel or not use it. And data-dependent control flow in the middle of the network, a dynamic top-k or a threshold-based selection like RT-DETR's `node_select`, forces a graph break and stays on the host. A detector built to this brief would keep more than 95% of its graph in one engine, and the obvious next step on a T4-class GPU is INT8 post-training quantisation, which targets the same convolution and GEMM kernels as FP16 and is expected to add roughly another 1.5× to 2×. Building and measuring such a network is the natural continuation of this work.

13 Conclusion

Profiling 24 detectors from five families at the operator level on one Tesla T4 gives three findings that hold across families. Convolution is the backbone bottleneck everywhere (56% to 90%) and runs in FP32 with the Tensor Cores idle, so mixed precision is the largest single lever. Normalisation is the clearest free win: the DETR family loses 28% to 32% of its backbone to a FrozenBatchNorm that folds into the convolution at zero accuracy cost. And a single un-compiled operator makes Deformable DETR the slowest model at 263 ms: an operator, not a parameter count, sets latency.

The optimisation rounds confirm the diagnosis at the sub-module level. The gains come from three mechanisms, in order: precision moves the dense convolution and GEMM operators by up to 6.6× on the Tensor Cores; fusion deletes the bandwidth-bound activations and norms by folding them into the convolution, taking SiLU to 4.9×; and engine capture collapses hundreds of per-image kernel launches into one, up to 11.5× fewer on RT-DETR R101. TensorRT FP16 reaches 5.06× and 5.57× on RT-DETR R50/R101, 1.78× to 2.88× on YOLO-S, and 2.32× on EfficientDet, all with mAP unchanged; `compile`+AMP reaches 3.2× to 3.4× on the DETR family, where it also overturned the round-one rule once CUDA graphs removed the head-Linear launch penalty. The floor is equally consistent: selection, NMS, RoIAlign, and the deformable-attention fallback are data-dependent or un-compiled, and no precision or fusion lever reaches them.

The transversal ranking of fast operations turns directly into a design brief (Section 12): a future detector assembled from the best-accelerating blocks, a fused Conv-BN-SiLU or MB-Conv backbone, a static neck, a dense NMS-free head, and GPU-resident shapes throughout, would keep more than 95% of its graph in one engine and be fast by construction rather than by retrofit. INT8 on the same convolution and GEMM kernels is the next expected 1.5× to 2×. Building and measuring that network, along with the compiled MSDA kernel and INT8 calibration, is the natural continuation of this study.

References

[1] Jimmy Lei Ba, Jamie Ryan Kiros, and Geoffrey E. Hinton. Layer normalization. *arXiv:1607.06450*, 2016.

[2] Nicolas Carion, Francisco Massa, Gabriel Synnaeve, Nicolas Usunier, Alexander Kirillov, and Sergey Zagoruyko. End-to-end object detection with transformers. In *ECCV*, 2020.

[3] Tri Dao, Daniel Y. Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. FlashAttention: Fast and memory-efficient exact attention with io-awareness. In *NeurIPS*, 2022.

[4] Xiaohan Ding, Xiangyu Zhang, Ningning Ma, Jungong Han, Guiguang Ding, and Jian Sun. Repvgg: Making vgg-style convnets great again. In *CVPR*, 2021.

[5] Zheng Ge, Songtao Liu, Feng Wang, Zeming Li, and Jian Sun. Yolox: Exceeding yolo series in 2021. In *arXiv:2107.08430*, 2021.

[6] Kaiming He, Georgia Gkioxari, Piotr Dollár, and Ross Girshick. Mask r-cnn. In *ICCV*, 2017.

[7] Dan Hendrycks and Kevin Gimpel. Gaussian error linear units (gelus). *arXiv:1606.08415*, 2016.

[8] Andrew G. Howard, Menglong Zhu, Bo Chen, Dmitry Kalenichenko, Weijun Wang, Tobias Weyand, Marco Andreetto, and Hartwig Adam. MobileNets: Efficient convolutional neural networks for mobile vision applications. *arXiv:1704.04861*, 2017.

[9] Sergey Ioffe and Christian Szegedy. Batch normalization: Accelerating deep network training by reducing internal covariate shift. In *ICML*, 2015.

[10] Andrew Lavin and Scott Gray. Fast algorithms for convolutional neural networks. In *CVPR*, 2016.

[11] Tsung-Yi Lin, Priya Goyal, Ross Girshick, Kaiming He, and Piotr Dollár. Focal loss for dense object detection. In *ICCV*, 2017.

[12] Shilong Liu, Feng Li, Hao Zhang, Xiao Yang, Xianbiao Qi, Hang Su, Jun Zhu, and Lei Zhang. Dab-detr: Dynamic anchor boxes are better queries for detr. In *ICLR*, 2022.

[13] Depu Meng, Xiaokang Chen, Zejia Fan, Gang Zeng, Houqiang Xu, Lu Yuan, Lei Sun, and Jingdong Wang. Conditional detr for fast training convergence. In *ICCV*, 2021.

[14] Prajit Ramachandran, Barret Zoph, and Quoc V. Le. Searching for activation functions. *arXiv:1710.05941*, 2017.

[15] Joseph Redmon, Santosh Divvala, Ross Girshick, and Ali Farhadi. You only look once: Unified, real-time object detection. In *CVPR*, 2016.

[16] Shaoqing Ren, Kaiming He, Ross Girshick, and Jian Sun. Faster r-cnn: Towards real-time object detection with region proposal networks. *NeurIPS*, 2015.

[17] Mingxing Tan, Ruoming Pang, and Quoc V. Le. EfficientDet: Scalable and efficient object detection. In *CVPR*, 2020.

[18] Zhi Tian, Chunhua Shen, Hao Chen, and Tong He. FCOS: Fully convolutional one-stage object detection. In *ICCV*, 2019.

[19] Ultralytics. Yolo11 architecture (c3k2, sppf, c2psa). `https://docs.ultralytics.com`, 2024.

[20] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *NeurIPS*, 2017.

[21] Ao Wang, Hui Chen, Lihao Liu, Kai Chen, Zijia Lin, Jungong Han, and Guiguang Ding. Yolov10: Real-time end-to-end object detection. In *NeurIPS*, 2024.

[22] Chien-Yao Wang, Alexey Bochkovskiy, and Hong-Yuan Mark Liao. Yolov7: Trainable bag-of-freebies sets new state-of-the-art for real-time object detectors. In *CVPR*, 2023.

[23] Chien-Yao Wang, Hong-Yuan Mark Liao, Yueh-Hua Wu, Ping-Yang Chen, Jun-Wei Hsieh, and I-Hau Yeh. CSPNet: A new backbone that can enhance learning capability of cnn. In *CVPRW*, 2019.

[24] Chien-Yao Wang, I-Hau Yeh, and Hong-Yuan Mark Liao. Yolov9: Learning what you want to learn using programmable gradient information. In *ECCV*, 2024.

[25] Xianzhe Xu, Yiqi Jiang, Weihua Chen, Yilun Huang, Yuan Zhang, and Xiuyu Sun. Damo-yolo: A report on real-time object detection design. In *arXiv:2211.15444*, 2022.

[26] Yian Zhao, Wenyu Lv, Shangliang Xu, Jinman Wei, Guanzhong Wang, Qingqing Dang, Yi Liu, and Jie Chen. Detrs beat yolos on real-time object detection. In *CVPR*, 2024.

[27] Xizhou Zhu, Weijie Su, Lewei Lu, Bin Li, Xiaogang Wang, and Jifeng Dai. Deformable detr: Deformable transformers for end-to-end object detection. In *ICLR*, 2021.